

HormuzWatch Production Readiness and Cloud Engineering Review Prompt

You are a Principal Cloud Architect, Staff Software Engineer, DevOps Engineer, Security Engineer, MLOps Engineer, and Technical Writer.

Analyze the entire HormuzWatch codebase and produce a complete production-readiness assessment, implementation roadmap, architecture review, cloud deployment strategy, security review, and project management plan.

Do not make assumptions.

Inspect every directory, source file, dependency, configuration file, Dockerfile, Terraform module, workflow, frontend component, backend service, ML model, and document.

The goal is to transform the current MVP into a production-quality Azure-focused cloud engineering portfolio project.

Project Overview

HormuzWatch is a maritime intelligence and threat detection platform consisting of:

- Go Backend
- React Router v7 Frontend
- Python ML Service
- WebSocket Hub
- Threat Aggregation Engine
- Geospatial Dashboard
- Real-Time Telemetry Processing
- Anomaly Detection Engine

Current status:

- Core functionality exists
 - Deployment not finalized
 - Terraform not implemented
 - CI/CD not implemented
 - SQLite migration to PostgreSQL in progress
 - Azure production deployment planned
 - Render used for development environments
 - Vercel used for frontend deployment
-

Deliverables Required

Produce the following outputs:

1. Executive Summary

Provide:

- Current maturity level
 - Production readiness score
 - Major risks
 - Deployment blockers
 - Architecture weaknesses
 - Security concerns
 - Technical debt summary
-

2. Project Roadmap

Create:

- Phase 1 (Critical)
- Phase 2 (High Priority)
- Phase 3 (Medium Priority)
- Phase 4 (Future Enhancements)

For each phase include:

- Objective
 - Estimated effort
 - Dependencies
 - Acceptance criteria
-

3. Repository Structure Review

Analyze:

- Current structure
- Folder organization
- Naming conventions
- Package boundaries
- Backend architecture
- Frontend architecture
- ML architecture

Recommend:

- Simplified structure
 - Clean architecture patterns
 - Service separation
 - Domain-driven organization
-

4. Database Review

Analyze:

- Current SQLite implementation
- Migration requirements
- PostgreSQL readiness
- Query design
- Schema design
- Index strategy

Generate:

- Migration plan
- Rollback strategy
- Environment-specific configuration

Target:

Azure Database for PostgreSQL

5. Environment Strategy

Design:

Development Environment

- Render
- PostgreSQL
- GitHub Actions

Production Environment

- Azure Container Apps
- Azure PostgreSQL
- Azure Key Vault
- Azure Event Hubs
- Azure Blob Storage

- Application Insights

Branching Strategy

main

- Production

develop

- Development

feature/*

- Feature branches

release/*

- Release branches

hotfix/*

- Emergency fixes

Generate Git workflow recommendations.

6. Terraform Review

Generate:

Terraform architecture

Resources:

- Resource Groups
- Container Registry
- Container Apps
- PostgreSQL
- Key Vault
- Event Hubs
- Blob Storage
- Managed Identities
- Application Insights

Provide:

- Module structure

- Environment structure
 - Remote state strategy
 - Secret handling strategy
-

7. CI/CD Review

Design GitHub Actions workflows.

Required pipelines:

Frontend

- Test
- Build
- Deploy Vercel

Backend

- Test
- Lint
- Security Scan
- Docker Build
- Registry Push
- Deploy

ML

- Test
- Build
- Deploy

Infrastructure

- Terraform Validate
- Terraform Plan
- Terraform Apply

Include:

- Branch protections
 - Release workflows
 - Rollback workflows
-

8. Security Review

Analyze:

Authentication

Authorization

Secrets Management

JWT Implementation

Admin Access

WebSocket Security

API Security

Container Security

Dependency Security

Generate:

Threat model

Risk matrix

Mitigation plan

9. Admin Portal Review

Requirements:

- Create User
- Update User
- Delete User
- Role Assignment
- Audit Logging
- Session Tracking
- Active User Monitoring

Remove:

- Hardcoded admin accounts

Replace with:

- Database-backed administration

Generate implementation design.

10. WebSocket Review

Evaluate:

Current implementation

Required additions:

- Real-time notifications
- Session tracking
- Admin monitoring
- Threat notifications
- Connection metrics

Provide:

Architecture diagram

Scaling strategy

Security controls

11. Plugin Architecture

Design plugin system for:

AIS Providers

Flight Tracking Providers

Weather Providers

Intelligence Sources

Satellite Sources

Future Data Sources

Requirements:

- Hot-pluggable architecture
- Provider abstraction
- Standard interface definitions
- Dynamic registration

Generate Go interface design.

12. Machine Learning Review

Evaluate:

Current model

Feature engineering

Data pipeline

Training process

Inference process

Future Azure integration

Generate:

Improvement roadmap

Model versioning strategy

Model registry strategy

Inference architecture

Performance optimization plan

13. Frontend Review

Evaluate:

React Router v7 structure

State management

Current Zustand usage

Component architecture

Routing strategy

Map rendering

WebSocket integration

Generate:

Refactoring recommendations

State architecture

Folder structure

Performance improvements

Accessibility review

14. Documentation Review

Generate missing documentation:

- Architecture
- Deployment
- Development Guide
- API Documentation
- Database Design
- Terraform Guide
- CI/CD Guide
- Runbooks
- Incident Response
- Monitoring Guide

- Security Guide
 - Contributor Guide
-

15. User Experience Review

Requirements:

Landing Page

Project Overview

Disclaimer Page

Terms and Conditions

Privacy Policy

Documentation Portal

Admin Portal

User Dashboard

Threat Dashboard

Generate:

UX improvement recommendations

Navigation structure

Content requirements

16. Reporting and Analytics

Design:

Threat Reports

Risk Reports

Operator Reports

Executive Summaries

PDF Generation

Export Formats

Dashboard Analytics

Generate implementation plan.

17. Versioning Strategy

Design:

Application Versioning

API Versioning

Database Versioning

ML Model Versioning

Terraform Versioning

Container Versioning

Release Tagging

Provide examples.

18. Monitoring and Operations

Design:

Logging

Metrics

Tracing

Alerting

Health Checks

SLIs

SLOs

Runbooks

Incident Response

Disaster Recovery

Generate operational procedures.

19. Cloud Engineering Portfolio Assessment

Evaluate the project from the perspective of:

- Azure Cloud Engineer
- DevOps Engineer
- Platform Engineer
- SRE
- Security Engineer

Identify:

- Missing Azure services
- Missing cloud engineering concepts
- Missing automation
- Missing observability
- Missing security controls

Recommend improvements that maximize portfolio and interview value.

20. Final Output

Generate:

1. Prioritized backlog
2. Architecture diagram
3. Technical debt register
4. Security findings
5. Deployment readiness checklist
6. Production launch checklist

7. Interview talking points
8. Estimated completion effort
9. Suggested GitHub Issues
10. Suggested project milestones

Provide all findings in a structured and actionable format.